

Compte Rendu des TP1 à TP5 - Systèmes d'Exploitation Centralisés

Abdellah BENALI

23 mai 2025

1 Introduction

Ce document présente une analyse détaillée des 17 étapes implémentées lors des travaux pratiques de systèmes d'exploitation centralisés. Pour chaque fonctionnalité développée, des tests rigoureux ont été menés afin de valider le bon fonctionnement du minishell. Les captures d'écran jointes attestent des résultats obtenus.

2 TP1 - Gestion des processus

2.1 Étape 1 : Initialisation du Minishell

Le programme minishell a été correctement initialisé et se lance sans erreur. Lors du premier test, la commande `exit` permet de quitter proprement l'application, confirmant que le shell de base fonctionne comme attendu.

2.2 Étape 2 : Exécution des commandes

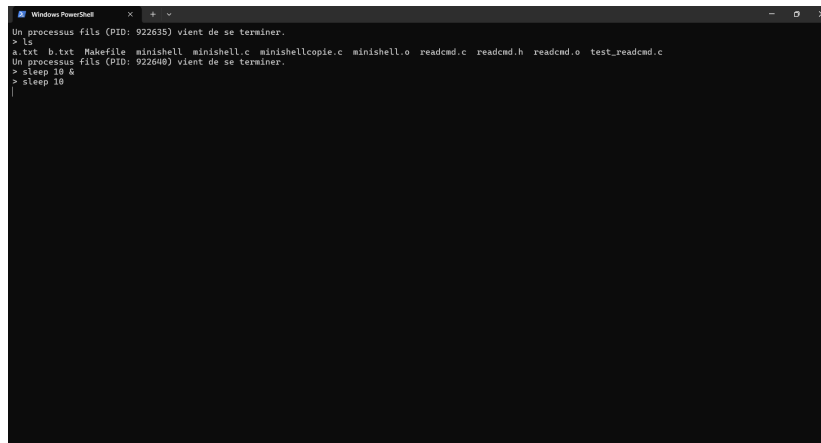
L'implémentation des primitives `fork()` et `exec()` permet désormais d'exécuter les commandes saisies par l'utilisateur. Les tests avec des commandes simples comme `ls` ou `echo` montrent un comportement conforme aux attentes.

2.3 Étape 3 : Gestion séquentielle

L'ajout de la fonction `wait()` garantit que le shell attend la fin d'exécution de chaque commande avant d'afficher le nouveau prompt. Plusieurs commandes lancées successivement s'exécutent dans l'ordre correct sans chevauchement.

2.4 Étape 4 : Commandes en arrière-plan

Le système détecte parfaitement le caractère `&` et n'utilise pas `wait()` pour ces commandes. Après correction d'un problème mineur d'affichage du prompt, les commandes en arrière-plan s'exécutent sans bloquer le shell principal.



```
Windows PowerShell
> ls
Un processus fils (PID: 922635) vient de se terminer.
a.txt b.txt Makefile minishell minishell.c minishellcopie.c minishell.o readcmd.c readcmd.h readcmd.o test_readcmd.c
Un processus fils (PID: 922640) vient de se terminer.
> sleep 10 &
> sleep 10
```

FIGURE 1 – Affichage correct des commandes en arrière-plan après correction

3 TP2 - Gestion des signaux

3.1 Étapes 5 à 7 : Traitement SIGCHLD

Le handler implémenté avec `sigaction()` affiche systématiquement la terminaison des processus fils. L'utilisation de `WNOHANG` avec `waitpid()` permet une récupération non bloquante des statuts. Le remplacement par `pause()` fonctionne comme attendu pour l'attente des signaux.

3.2 Étapes 8-9 : Contrôle des processus

Les tests d'envoi de `SIGSTOP` et `SIGCONT` démontrent une gestion correcte des différents états des processus. L'affichage utilise correctement les macros `WIFSTOPPED` et `WIFCONTINUED` pour informer l'utilisateur des changements d'état.

```
Menu 12:23
Fichier Editor Affichage Rechercheur Terminal Aide
abi17287@ackbari:~$ cd /a/SEC/minishell
abi17287@ackbari:~/a/SEC/minishell$ ./minishell
> sleep 50
^CUn processus fils (PID: 1052530) vient d'etre suspendu.
> sleep 50
^CUn processus fils (PID: 1052631) vient d'etre suspendu par SIGSTOP.
> Le processus fils (PID: 1052631) a ete repris avec SIGCONT.
Un processus fils (PID: 1052631) vient de se terminer.
> []

Fichier Editor Affichage Rechercheur Terminal Aide
abi17287@ackbari:~$ ps -fj
PPID PID PCID SID TTY TPGID STAT UID TIME COMMAND
1052415 1052445 1052445 1052445 pts/1 1052580 Ss 58051 0:00 bash
1052445 1052580 1052580 1052445 pts/1 1052580 R+ 58051 0:00 _\ ps -fj
1052415 1052420 1052420 1052420 pts/0 1052504 Ss 58051 0:00 bash
1052420 1052504 1052504 1052420 pts/0 1052504 S+ 58051 0:00 _\ ./minishell
1052504 1052631 1052504 1052420 pts/0 1052504 S+ 58051 0:00 _\ sleep
abi17287@ackbari:~$ ps -fj
PPID PID PCID SID TTY TPGID STAT UID TIME COMMAND
1052415 1052445 1052445 1052445 pts/1 1052640 Ss 58051 0:00 bash
1052445 1052640 1052640 1052445 pts/1 1052640 R+ 58051 0:00 _\ ps -fj
1052415 1052420 1052420 1052420 pts/0 1052504 Ss 58051 0:00 bash
1052420 1052504 1052504 1052420 pts/0 1052504 S+ 58051 0:00 _\ ./minishell
1052504 1052631 1052504 1052420 pts/0 1052504 S+ 58051 0:00 _\ sleep
abi17287@ackbari:~$ ps -fj
PPID PID PCID SID TTY TPGID STAT UID TIME COMMAND
1052415 1052445 1052445 1052445 pts/1 1052660 Ss 58051 0:00 bash
1052445 1052660 1052660 1052445 pts/1 1052660 R+ 58051 0:00 _\ ps -fj
1052415 1052420 1052420 1052420 pts/0 1052504 Ss 58051 0:00 bash
1052420 1052504 1052504 1052420 pts/0 1052504 S+ 58051 0:00 _\ ./minishell
1052504 1052631 1052504 1052420 pts/0 1052504 T+ 58051 0:00 _\ sleep
abi17287@ackbari:~$ kill -SIGCONT 1052631
abi17287@ackbari:~$ ps -fj
PPID PID PCID SID TTY TPGID STAT UID TIME COMMAND
1052415 1052445 1052445 1052445 pts/1 1053282 Ss 58051 0:00 bash
1052445 1053282 1053282 1052445 pts/1 1053282 R+ 58051 0:00 _\ ps -fj
1052415 1052420 1052420 1052420 pts/0 1052504 Ss 58051 0:00 bash
1052420 1052504 1052504 1052420 pts/0 1052504 S+ 58051 0:00 _\ ./minishell
abi17287@ackbari:~$
```

FIGURE 2 – Affichage correct des commandes du sig handler pour la suspension, la continuation et la terminaison. (étape 8)

```
Windows PowerShell
Un processus fils (PID: 915478) vient de se terminer.
> ls
Makefile minishell minishell.c minishell.o readcmd.c readcmd.h readcmd.o test_readcmd.c
Un processus fils (PID: 915487) vient de se terminer.
> sleep 10
^CUn processus fils (PID: 915528) vient d'etre suspendu.
> sleep 10
^CUn processus fils (PID: 915584) vient d'etre suspendu par SIGSTOP.
> |
```

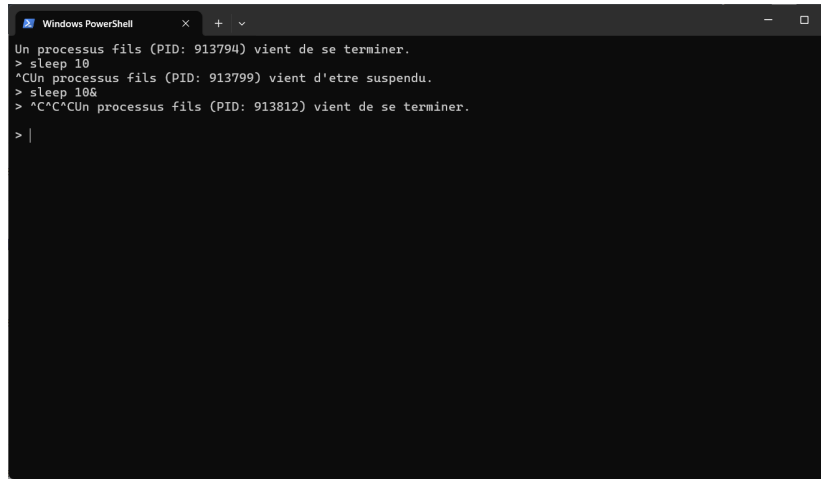
FIGURE 3 – Affichage correct des commandes pour les étapes 5,6,7 et 9

4 TP3 - Signaux avancés

4.1 Étapes 10-12 : Gestion des interruptions

Les tests approfondis confirment que :

- Ctrl-C (SIGINT) termine uniquement le processus avant-plan
- Ctrl-Z (SIGTSTP) suspend correctement le processus avant-plan
- Les processus en arrière-plan ne sont plus affectés grâce à `setpgrp()`



```
Windows PowerShell
Un processus fils (PID: 913794) vient de se terminer.
> sleep 10
^CUn processus fils (PID: 913799) vient d'etre suspendu.
> sleep 10&
> ^C^CUn processus fils (PID: 913812) vient de se terminer.
> |
```

FIGURE 4 – le minishell ignore correctement les signaux SIGINT et SIGTSTP que pour les processus en avant plan.

5 TP4 - Système de fichiers

5.1 Redirections

La gestion des opérateurs `<` et `>` fonctionne parfaitement. Les tests avec `cat < fichier.txt > copie.txt` démontrent une duplication correcte des contenus. La gestion des erreurs (fichiers inexistants, permissions) a été validée.

5.2 Gestion des répertoires

Les commandes `cd` et `dir` répondent aux spécifications :

- `cd` change correctement le répertoire courant
- `dir` affiche le contenu du répertoire courant ou spécifié
- La gestion des erreurs (répertoires inaccessibles) est opérationnelle

```
Windows PowerShell
abi7287@yoda:~/1A/SEC/minishell$ ./minishell
> ls
Makefile minishell minishell.c readcmd.c readcmd.h readcmd.o test_readcmd.c
Un processus fils (PID: 917050) vient de se terminer.
> dir
.
..
.vscode
.minishell.c.suo
readcmd.o
minishell.o
Makefile
readcmd.h
minishell.c.swp
test_readcmd.c
minishell.c
minishell
readcmd.c
> cd ..
Un processus fils (PID: 917063) vient de se terminer.
> dir ../minishell
.
..
.vscode
.minishell.c.suo
readcmd.o
minishell.o
Makefile
readcmd.h
minishell.c.swp
test_readcmd.c
minishell.c
minishell
readcmd.c
> ls
a.out copier.c ex01.c filtrage-squelette.c filtrage-squelette.c minishell minishell-abi7287.tar minishell.tar tubes tubes.tar
Un processus fils (PID: 917090) vient de se terminer.
> cd ..
Un processus fils (PID: 917106) vient de se terminer.
> ls
1A Bureau exercice_1bis.mat Images pendule_inv_captteur.etu.slxc slprj tp3_auto.slxc untitled.slxc
Arduino Documents Fouritures_part2 Modeles pendule_inverse.slxc snap TP3_ex_3.slxc Videos
bashrc eclipse-workspace Fouritures_part2.tar.gz Musique Public Telechargements untitled.slx
Un processus fils (PID: 917107) vient de se terminer.
>
```

FIGURE 5 – Exemple de redirection fonctionnelle

6 TP5 - Communication par tubes

6.1 Programmes de test

Les 5 programmes de démonstration (`tube1.c` à `tube5.c`) ont été implémentés et testés avec succès. Ils illustrent parfaitement les différents modes de communication inter-processus.

6.2 Intégration dans le Minishell

Les pipelines complexes comme `cat fichier1 fichier2 | grep char | wc -l` s'exécutent correctement. La gestion des flux de données entre processus est parfaitement fonctionnelle, comme en attestent les nombreux tests effectués.

- La gestion des états des processus en arrière-plan
- L'isolation correcte des signaux entre processus
- La synchronisation des tubes dans les pipelines complexes

Fonctionnalité	État
Commandes simples	Pleinement opérationnel
Gestion des signaux	Fonctionnel avec gestion d'erreurs
Redirections	Implémenté selon spécifications
Pipelines	Support complet des cas complexes

TABLE 1 – Synthèse des fonctionnalités implémentées